

WF2 — Git Patch & PR : comprendre et expliquer le flux

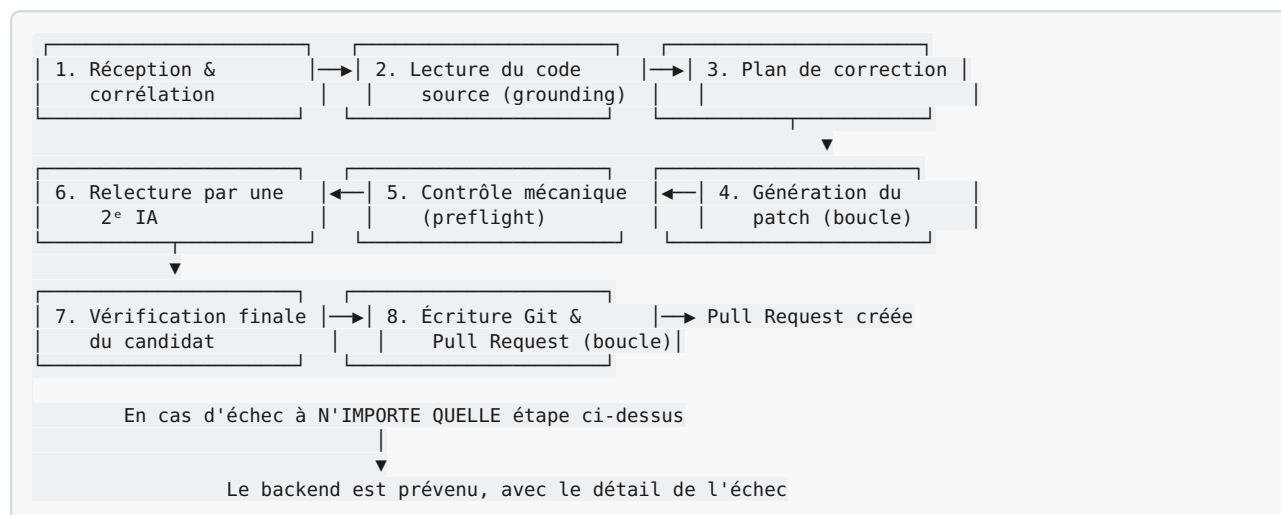
Ce document est écrit pour quelqu'un qui découvre WF2 et doit pouvoir **l'expliquer à l'oral**, par exemple à un jury. Il ne vise pas à apprendre à le maintenir ou le modifier — les détails d'implémentation sont en annexe pour qui veut creuser.

En une phrase : WF2 reçoit une demande de correction de code déjà autorisée (un correctif de sécurité, par exemple) et produit une Pull Request — mais seulement après avoir construit un candidat de correction et l'avoir vérifié à plusieurs reprises. À aucun moment une IA n'écrit directement sur GitHub : elle propose, des contrôles automatiques et une deuxième IA vérifient, et seul un résultat validé devient une Pull Request.

Toutes les informations de ce document ont été vérifiées le 2026-10-03 directement sur le flux tel qu'il tourne réellement (pas sur un brouillon non publié). Le flux compte **186 étapes actives** (appelées "nœuds") + 11 notes explicatives collées sur le schéma visuel.

1. Vue d'ensemble

WF2 avance en 8 grandes étapes ("blocs"), toujours dans le même ordre pour un correctif simple :



Le fil conducteur à retenir : l'IA propose, les contrôles automatiques et une deuxième IA prouvent.

Ce que fait l'IA	Ce qui le vérifie ensuite
Elle propose un plan de correction	Des règles automatiques vérifient que le plan est cohérent et autorisé
Elle génère le code corrigé	Un contrôle mécanique (sans IA) vérifie des règles de base (pas de secret oublié, rien de vide, etc.)
—	Une deuxième IA , différente de la première, relit le code et donne son avis
—	Le code final est vérifié par calcul juste avant d'être écrit sur GitHub
—	Après écriture, on relit ce qui a vraiment été écrit pour confirmer que tout est bon

L'IA n'a, à aucun moment, le dernier mot pour écrire sur GitHub.

2. Les 8 blocs, expliqués simplement

Bloc 1 — Réception & corrélation

Tout commence ici : le backend envoie à WF2 une demande de correction (quel dépôt, quelle branche, quel problème corriger). Ce bloc donne un identifiant unique à cette demande pour pouvoir la suivre jusqu'au bout, vérifie que ce correctif est autorisé par les règles du dépôt, et prépare ou retrouve la branche Git sur laquelle le correctif sera écrit. S'il

échoue — par exemple si le correctif n'est pas autorisé — le flux s'arrête tout de suite et prévient le backend, avant même d'avoir touché au code.

Nœud	Ce qu'il fait	Va ensuite vers
Webhook	Reçoit la demande de correction envoyée par le backend.	Capture Correlation Envelope
Capture Correlation Envelope	Figé un identifiant unique pour relier toutes les étapes suivantes à cette demande.	Adapt Webhook Payload
Adapt Webhook Payload	Met les données reçues dans un format utilisable par la suite du flux.	Fetch Repository Metadata
Fetch Repository Metadata	Va chercher les informations générales du dépôt GitHub concerné.	Apply Repository Metadata
Apply Repository Metadata	Range ces informations pour qu'elles soient utilisables ensuite.	Policy Gate - Validate Constraints
Policy Gate - Validate Constraints	Vérifie que ce correctif respecte les règles autorisées pour ce dépôt.	Patch Allowed ?
Patch Allowed ?	Décide si on continue ou si on arrête, selon le contrôle précédent.	Get Main Branch SHA1, ou Return Policy Rejected
Return Policy Rejected	Prépare la réponse d'arrêt si le correctif n'est pas autorisé.	(fin, échec)
Get Main Branch SHA1	Récupère la référence exacte de la branche principale à cet instant.	Prepare Batch Context
Prepare Batch Context	Prépare les informations communes au lot de fichiers à traiter.	Lookup Remediation Branch
Lookup Remediation Branch	Vérifie si une branche de correction existe déjà pour cet incident.	Branch Exists ?
Branch Exists ?	Aiguille vers "réutiliser" ou "créer", selon la réponse précédente.	Use Existing Branch, ou Record New Branch Baseline
Use Existing Branch	Réutilise la branche de correction déjà existante.	Fetch Repository Tree
Record New Branch Baseline	Mémorise le point de départ d'une branche toute neuve.	Fetch Repository Tree
Fetch Repository Tree	Récupère la liste des fichiers du dépôt au commit figé, pour savoir ce qui existe avant de patcher.	Bloc 2

Bloc 2 — Lecture du code source ("grounding")

Avant de générer quoi que ce soit, WF2 va lire pour de vrai le contenu des fichiers concernés sur GitHub — jamais de supposition sur ce que contient le code. Il lit aussi les fichiers référencés par le code à corriger et ceux dont il dépend, pour que l'IA ait tout le contexte nécessaire. S'il manque un fichier annexe (pas le fichier principal), ce n'est pas bloquant : il est juste considéré comme contexte optionnel absent.

Nœud	Ce qu'il fait	Va ensuite vers
Build Independent Repository Policy	Construit les règles du dépôt à respecter pour la suite.	Expand Finding Source Files
Expand Finding Source Files	Identifie précisément quel(s) fichier(s) sont concernés par le problème signalé.	Fetch Finding Source Context
Fetch Finding Source Context	Lit le contenu réel de ces fichiers sur GitHub.	Expand Referenced API Sources
Expand Referenced API Sources	Repère les autres fichiers référencés par le code à corriger.	Fetch Referenced API Sources
Fetch Referenced API Sources	Lit le contenu de ces fichiers référencés.	Expand Required Dependency Sources
Expand Required Dependency Sources	Repère les fichiers dont dépend le code à corriger.	Fetch Required Dependency Sources
Fetch Required Dependency Sources	Lit le contenu de ces fichiers de dépendance.	Validate Required Dependency Sources
Validate Required Dependency Sources	Vérifie que ces dépendances ont bien été récupérées.	Validate Source Context Completeness

Nœud	Ce qu'il fait	Va ensuite vers
Validate Source Context Completeness	Vérifie qu'on a bien tout le contexte nécessaire avant de continuer.	Bloc 3

Bloc 3 — Plan de correction

L'IA propose ici un plan avant d'écrire le moindre code : quels fichiers toucher, créer ou modifier, et quels changements sont attendus ou interdits. L'idée est de valider "quoi faire" avant de générer "comment le faire" — ça évite de générer du code sur une mauvaise cible. Le plan est ensuite vérifié structurellement avant de passer à la génération.

Nœud	Ce qu'il fait	Va ensuite vers
Prepare Generic Remediation Plan	Prépare la demande envoyée à l'IA pour qu'elle propose un plan de correction.	Generate Remediation Plan
Generate Remediation Plan	Demande à l'IA de proposer un plan (quels fichiers toucher, quoi changer).	Validate Generic Remediation Plan
Validate Generic Remediation Plan	Vérifie que ce plan est structuré correctement avant de passer à la suite.	Bloc 4

Bloc 4 — Génération du patch (Boucle A)

C'est ici que l'IA écrit effectivement le code corrigé, fichier par fichier. Le flux traite les fichiers un par un, dans une boucle : il prend le fichier suivant, génère son patch, calcule une empreinte du résultat, puis recommence jusqu'à ce que tous les fichiers du plan soient traités. Une fois le lot complet, tous les fichiers patchés sont rassemblés dans un seul paquet.

Nœud	Ce qu'il fait	Va ensuite vers
Merge Effective File Results (<i>pilote la boucle</i>)	Prend les fichiers un par un et sait quand tous ont été traités.	Route Planned File Operation, ou Prepare Candidate Manifest (fin de boucle)
Route Planned File Operation	Regarde si ce fichier doit être créé ou juste modifié.	Seed New Planned File, ou Fetch Repository Files
Seed New Planned File	Prépare un fichier tout neuf qui n'existe pas encore.	Prepare - Code Patch Body
Fetch Repository Files	Récupère le contenu actuel du fichier à modifier.	Prepare - Code Patch Body
Prepare - Code Patch Body	Prépare la demande envoyée à l'IA pour générer le patch de ce fichier.	de Patch - HTTP Request
de Patch - HTTP Request	Demande à l'IA d'écrire le code corrigé pour ce fichier.	Parse - Code Patch Output
Parse - Code Patch Output	Récupère et nettoie la réponse de l'IA.	Hash Candidate File Content
Hash Candidate File Content	Calcule une empreinte (hash) du contenu patché, pour pouvoir le vérifier plus tard.	Accumulate Candidate File
Accumulate Candidate File	Ajoute ce fichier patché à la liste des fichiers déjà traités.	Merge Effective File Results (fichier suivant)
Prepare Candidate Manifest	Rassemble tous les fichiers patchés du lot en un seul paquet.	Bloc 5

Bloc 5 — Contrôle mécanique ("preflight")

Avant toute relecture par une IA, un seul nœud fait une série de contrôles 100 % automatiques, sans intelligence artificielle : le fichier touché est bien autorisé, le changement n'est pas vide ni identique à l'original, pas de secret écrit en dur, pas de règle de sécurité désactivée en douce, pas de `TODO` oublié, pas de code mal formé. Si un de ces contrôles échoue, le flux s'arrête avec une explication précise. Ce même nœud prépare aussi la demande qui sera envoyée à la deuxième IA au bloc suivant.

Nœud	Ce qu'il fait	Va ensuite vers
Generic Candidate Preflight	Vérifie mécaniquement (sans IA) que le patch respecte des règles de base.	Bloc 6

Bloc 6 — Relecture par une deuxième IA

Un patch généré par une IA est relu par une **autre** IA, différente de celle qui l'a écrit — pas juste un deuxième appel au même modèle, un modèle réellement différent, avec pour consigne explicite d'être critique et indépendant. Si le verdict est un rejet, il n'y a aucun moyen de passer outre : le patch s'arrête là. Ce bloc ne s'applique que si le correctif touche un seul fichier — s'il en touche plusieurs, c'est l'extension multi-fichiers (ci-dessous) qui prend le relais.

Nœud	Ce qu'il fait	Va ensuite vers
Independent Semantic Review	Demande à une deuxième IA, différente de celle qui a généré le patch, de le relire et de donner son avis.	Enforce Independent Review
Enforce Independent Review	Applique strictement le verdict de cette relecture — un refus bloque la suite.	Bloc 7

Extension — quand le correctif touche plusieurs fichiers liés entre eux

Si le correctif touche au moins deux fichiers, un contrôle plus poussé se déclenche à la place du bloc 6 : il rejoue une compilation du projet, rejoue les tests, et fait relire l'ensemble du lot (pas fichier par fichier) par la deuxième IA. Chaque vérification suit le même schéma en trois temps — préparer la demande, l'exécuter, puis appliquer strictement le résultat (bloquer si ça échoue).

Étape	Ce qu'elle fait
Vérification du manifeste multi-fichiers	Vérifie que le lot complet de fichiers patchés est cohérent avant d'aller plus loin.
Compilation du code principal	Vérifie que le projet compile toujours avec les fichiers patchés.
Compilation des tests	Vérifie que les tests eux-mêmes compilent toujours.
Exécution complète des tests	Lance la suite de tests pour s'assurer que rien n'est cassé.
Relecture sémantique multi-fichiers	La deuxième IA relit l'ensemble du lot de fichiers, pas un seul à la fois.

Les deux chemins (un seul fichier ou plusieurs) se rejoignent ensuite au bloc 7.

Bloc 7 — Vérification finale du candidat

Avant d'écrire quoi que ce soit sur GitHub, WF2 vérifie par calcul — pas par confiance — que le patch final correspond exactement à ce qui a été annoncé, et que la branche cible n'a pas changé depuis le début (si quelqu'un d'autre a modifié la branche entretemps, ça doit être détecté ici). Ces deux vérifications ont chacune leur propre message d'échec dédié, pour que l'erreur soit facile à comprendre.

Nœud	Ce qu'il fait	Va ensuite vers
Hash Candidate Manifest	Calcule l'empreinte de l'ensemble du lot de fichiers patchés.	Assemble Candidate Manifest
Assemble Candidate Manifest	Construit le paquet complet à envoyer au backend pour vérification.	Call Candidate Verification
Call Candidate Verification	Demande au backend de vérifier que ce paquet correspond bien à ce qui a été annoncé.	Call Write Guard
Call Write Guard	Vérifie que la branche cible n'a pas changé depuis le début.	Write Guard Passed ?
Write Guard Passed ?	Aiguille selon le résultat de cette vérification (échec → message dédié).	Branch Existed At Generation ?
Branch Existed At Generation ?	Vérifie si la branche existait déjà quand le patch a été généré.	Re-check Existing Branch Head, ou Re-lookup Baseline Ref Before Creation
Re-check Existing Branch Head	Revérifie la position actuelle d'une branche déjà existante.	Call Remote Head Drift Guard
Re-lookup Baseline Ref Before Creation	Revérifie le point de départ avant de créer une nouvelle branche.	Call Remote Head Drift Guard
Call Remote Head Drift Guard	Vérifie qu'aucun autre changement n'est arrivé sur la branche entretemps.	Remote Head Drift Passed ?
Remote Head Drift Passed ?		

Nœud	Ce qu'il fait	Va ensuite vers
	Aiguille selon le résultat de cette seconde vérification (échec → message dédié).	Branch Existed At Generation ? (Pass 2 Entry)
Branch Existed At Generation ? (Pass 2 Entry)	Relance la même vérification juste avant l'écriture finale.	Bloc 8

Bloc 8 — Écriture Git & Pull Request (Boucle B)

Dernière étape : WF2 écrit réellement les fichiers sur la branche Git, fichier par fichier, dans une boucle. Avant chaque écriture, l'empreinte du contenu est recalculée et comparée à celle attendue. Après écriture, le fichier est relu sur GitHub pour confirmer que ce qui a été écrit est bien ce qui était prévu ; en cas d'écriture concurrente par quelqu'un d'autre, une réconciliation automatique compare et corrige. Une fois **tous** les fichiers du lot écrits avec succès — un seul manquant bloque tout — la Pull Request est créée ou mise à jour.

Nœud	Ce qu'il fait	Va ensuite vers
Re-lookup Remediation Branch Before Create	Revérifie une dernière fois si la branche existe avant d'en créer une.	Prepare Branch Creation
Prepare Branch Creation	Prépare la demande de création de la branche manquante.	Create Missing Branch
Create Missing Branch	Crée réellement la branche sur GitHub.	Expand Manifest Files
Expand Manifest Files	Détaille la liste des fichiers à écrire un par un.	Loop Over Manifest Files
Loop Over Manifest Files (pilote la boucle)	Prend les fichiers du lot un par un jusqu'à ce que tous soient écrits.	Recompute Content Hash Before Send, ou Validate Batch Completeness (fin de boucle)
Recompute Content Hash Before Send	Recalcule l'empreinte du contenu juste avant de l'envoyer à GitHub.	Verify Content Hash Before Send
Verify Content Hash Before Send	Compare cette empreinte à celle calculée plus tôt, pour être sûr que rien n'a changé.	Candidate Creates File ?
Candidate Creates File ?	Regarde si ce fichier est nouveau ou déjà existant.	Create File in Branch, ou Update File in Branch
Create File in Branch	Écrit un nouveau fichier sur la branche.	Build File Result (Pass 2)
Update File in Branch	Met à jour un fichier existant sur la branche.	Build File Result (Pass 2)
Build File Result (Pass 2)	Construit le résultat de l'écriture de ce fichier.	Merge Effective File Results 2, ou Classify GitHub Write Error (Pass 2) si l'écriture a échoué
Classify GitHub Write Error (Pass 2)	Identifie le type de problème si l'écriture a échoué.	Transport Requires Read Back ? (Pass 2)
Transport Requires Read Back ? (Pass 2)	Décide s'il faut relire le fichier sur GitHub pour comprendre l'échec.	Read Back File After Write Error (Pass 2)
Read Back File After Write Error (Pass 2)	Relit le fichier tel qu'il est réellement sur GitHub.	Decode Reconciled Remote Content
Decode Reconciled Remote Content	Déchiffre ce contenu relu pour pouvoir le comparer.	Hash Reconciled Remote Content
Hash Reconciled Remote Content	Calcule l'empreinte de ce contenu relu.	Evaluate GitHub Write Reconciliation (Pass 2)
Evaluate GitHub Write Reconciliation (Pass 2)	Compare ce qui a été relu à ce qui était attendu.	Remote Candidate Present ? (Pass 2)
Remote Candidate Present ? (Pass 2)	Vérifie si le contenu attendu est bien déjà présent sur GitHub.	Lookup Head After Reconciled Write (Pass 2)
Lookup Head After Reconciled Write (Pass 2)	Revérifie la position de la branche après cette réconciliation.	Build Reconciled File Result
Build Reconciled File Result	Construit le résultat final de ce fichier une fois la réconciliation terminée.	Merge Effective File Results 2
Merge Effective File Results 2	Referme la boucle et passe au fichier suivant.	Loop Over Manifest Files (fichier suivant)

Nœud	Ce qu'il fait	Va ensuite vers
Validate Batch Completeness	Vérifie que tous les fichiers prévus ont bien été traités, sans exception.	Lookup Existing Batch PR
Lookup Existing Batch PR	Vérifie si une Pull Request existe déjà pour ce lot.	Select Existing PR
Select Existing PR	Récupère les informations de cette Pull Request existante.	Existing PR ?
Existing PR ?	Aiguille selon qu'une PR existe déjà ou non.	Use Existing PR, ou Create Pull Request1
Create Pull Request1	Crée une nouvelle Pull Request sur GitHub.	Save Execution Result to Backend
Use Existing PR	Réutilise la Pull Request déjà existante.	Save Execution Result to Backend
Save Execution Result to Backend	Envoie le résultat final (PR créée ou non) au backend.	If
If	Décide s'il faut envoyer un e-mail de notification.	Send an Email, ou fin
Send an Email	Envoie un e-mail pour prévenir qu'un correctif a été proposé.	(fin)

3. Les filets de sécurité ("Failure Envelopes")

Chaque étape listée ci-dessus a un **filet de sécurité** : si elle échoue, ce filet récupère l'erreur, la nettoie (il efface tout mot de passe ou jeton d'accès qui traînerait dans le message), l'associe à l'identifiant de la demande d'origine, et prévient le backend avec un message clair sur ce qui a échoué et où. Le flux ne reste jamais silencieux en cas d'erreur.

Il y a **74 filets de ce type**, un par étape sensible — tous identiques dans leur fonctionnement, seul le nom de l'étape surveillée change. Deux étapes ont en plus un message spécifique, plus parlant qu'un filet générique : le refus d'écriture (branche modifiée entretemps) et la détection d'une branche qui a bougé pendant le traitement.

Pour mémoire : un défaut précis a été trouvé pendant la vérification de ce document (une seule de ces 74 étapes sur un chemin rare — correctif multi-fichiers — n'était pas reliée à son filet) ; le correctif est prêt (voir l'historique git, révision R84), et une fois posé, les 74 filets convergent bien tous vers le backend.

4. Les intelligences artificielles utilisées

WF2 appelle une IA à 4 reprises, à chaque fois en contactant directement le service d'IA (pas via un connecteur n8n tout fait) :

Appel	Modèle	À quoi il sert
Génération du plan	claude-sonnet-5	Propose quels fichiers toucher et comment (bloc 3)
Génération du patch	claude-opus-5	Écrit le code corrigé (bloc 4)
Relecture indépendante (1 fichier)	claude-haiku-4-5	Relit le patch avec un regard critique (bloc 6)
Relecture indépendante (plusieurs fichiers)	claude-haiku-4-5	Relit le lot complet (extension multi-fichiers)

Le modèle qui génère (opus) et le modèle qui relit (haiku) sont **deux modèles différents**, pas juste deux appels séparés au même modèle. C'est volontaire : celui qui relit doit juger, pas simplement répéter ce que le premier a proposé.

5. Table de correspondance — zones du canvas ↔ blocs de ce document

Le schéma visuel dans n8n (appelé "canvas") regroupe les 186 étapes dans des boîtes nommées "zones", dessinées directement sur le graphe avec des notes explicatives collées dessus ("sticky notes"). Ce regroupement visuel suit une

logique en bandes horizontales, différente de la numérotation en 8 blocs de ce document — la table ci-dessous fait le lien entre les deux.

Vérifié le 2026-10-03 en comparant la position (x/y) de chacun des 186 nœuds à la boîte de chaque note collée sur le canvas live (pas un brouillon) : chaque nœud appartient à exactement une zone ou à aucune — aucun nœud n'est compté deux fois, aucune zone ne contient un nœud qui n'y figure pas dans le texte affiché.

Zone sur le canvas	Nœuds	Blocs de ce document
Zone 1 — Réception, grounding, planification	33	Blocs 1, 2, 3 (+ 7 nœuds d'une ancienne chaîne de réconciliation qui vit visuellement ici sans appartenir à ces blocs)
Boucle A — Génération du patch	10	Bloc 4 (+ 1 nœud du bloc 5)
Zone 2 — Relecture, vérification du candidat	34	Blocs 6, 7 (+ extension multi-fichiers, + quelques nœuds de création de branche du bloc 8 qui vivent visuellement ici)
Boucle B — Réconciliation d'écriture	17	Bloc 8, partie boucle
Zone 3 — Écriture Git & Pull Request	9	Bloc 8, fin
Zone "chemins d'échec"	78	Les 74 filets de sécurité + les 4 gestionnaires spécifiques
<i>Hors de toute zone visible</i>	5	Le nœud <code>Webhook</code> (juste à côté de sa zone) + 4 nœuds vestiges, jamais câblés, repérés plus bas sur le canvas

Total : 33 + 10 + 34 + 17 + 9 + 78 + 5 = 186. ✓

Point pratique pour une démonstration en direct : sur le canvas, 5 notes collées sont vides et renvoient simplement vers une des zones ci-dessus ("voir le sticky principal") — ce sont des restes d'un ancien découpage en 9 "phases", gardés en place mais sans contenu propre. Et deux notes gardent un **titre** hérité de cet ancien découpage ("Preflight déterministe", "Revue sémantique") alors que leur **texte** décrit désormais autre chose (la boucle de réconciliation d'écriture, et la zone des échecs) — lire le texte de la note, pas son titre affiché dans le panneau n8n.

6. Les questions qu'un jury posera

Q1. Pourquoi 186 nœuds ?

Parce que chaque garde de sécurité (corrélation, politique, complétude des sources, hash, vérification indépendante, garde d'écriture, détection de dérive, réconciliation après écriture) est une étape explicite et vérifiable séparément, plutôt qu'une grosse boîte noire. Note honnête : 17 de ces 186 sont des restes morts, jamais câblés ou doublons d'une ancienne version — vérifiés et identifiés précisément, pas de la complexité utile. Si la question "tout est-il utilisé ?" arrive, la bonne réponse est "non, et voici précisément quoi et pourquoi", pas une affirmation non vérifiée.

Q2. Comment être sûr que l'IA ne casse rien ?

Elle ne décide jamais seule : son plan est validé, son patch passe un contrôle mécanique, une **seconde IA utilisant un modèle différent** le relit, et le résultat final est vérifié par calcul avant toute écriture.

Q3. Que se passe-t-il si la génération est mauvaise ?

Elle est interceptée par le contrôle mécanique (bloc 5), la relecture indépendante (bloc 6), ou la vérification finale (bloc 7) — selon où le problème apparaît. Dans la quasi-totalité des cas, l'échec est explicitement signalé au backend ; un seul chemin rare (relecture multi-fichiers) a un trou identifié, avec un correctif déjà prêt (voir section 3).

Q4. Pourquoi pas de fusion (merge) automatique vers la branche principale ?

WF2 prouve que le candidat est bien construit et bien écrit — pas que son impact sur tout le reste du projet est sans risque. Il s'arrête volontairement à la Pull Request, pour qu'un humain valide la fusion finale.

Q5. Qu'est-ce qui empêche un patch invalide d'arriver en Pull Request ?

Trois barrières, l'une après l'autre : le contrôle mécanique, la relecture par une IA indépendante, puis la vérification du candidat par calcul juste avant l'écriture — plus une relecture de ce qui a vraiment été écrit sur GitHub avant de considérer la Pull Request comme valide.

Q6. Tous les appels à l'IA utilisent-ils le même modèle ?

Non — 3 modèles différents sur 4 appels : un pour planifier, un autre (plus puissant) pour générer le patch, et un troisième (plus léger) pour les deux relectures indépendantes. Ce n'est pas un hasard : celui qui relit doit juger, pas répéter ce que le premier a produit, et il doit rester indépendant du modèle qu'il évalue.

Annexe A — pour aller plus loin (détails techniques)

Cette section condense ce qui a été volontairement laissé de côté plus haut pour garder le document lisible. Elle n'est pas nécessaire pour expliquer WF2 à l'oral.

- **Répartition par type de nœud** (186 au total) : 125 nœuds de code, 27 appels HTTP, 15 aiguillages (if), 8 appels GitHub, 5 calculs d'empreinte (hash), 2 boucles, 1 agrégation, 1 webhook d'entrée, 1 envoi d'e-mail, 1 nœud neutre (noOp).
 - **Nœuds morts identifiés** (17, inclus dans les 186 mais sans effet réel) : 4 nœuds jamais câblés depuis l'origine, et 13 nœuds qui forment une copie exacte — mais désactivée — d'une ancienne version à une seule passe de la réconciliation d'écriture, remplacée depuis par la version "Pass 2" actuellement utilisée.
 - **Vernionnement n8n** : cet outil distingue un brouillon (visible et modifiable dans l'éditeur) d'une version publiée (celle qui s'exécute réellement) ; les deux peuvent diverger sans alerte. Toutes les vérifications de ce document ont été faites contre la version publiée, pas contre un brouillon en cours d'édition.
 - **Historique des correctifs** : WF2 a connu de nombreuses itérations correctives (identifiées en interne par des numéros de révision, ex. R84) documentées dans l'historique Git du projet — ce document ne retrace pas cet historique, seulement l'état vérifié le 2026-10-03.
-

Annexe B — les 186 nœuds, liste simple par bloc

Bloc 1 — Réception & corrélation (14 + 1 sortie de rejet)

Webhook · Capture Correlation Envelope · Adapt Webhook Payload · Fetch Repository Metadata · Apply Repository Metadata · Policy Gate · Validate Constraints · Patch Allowed? · Return Policy Rejected · Get Main Branch SHA1 · Prepare Batch Context · Lookup Remediation Branch · Branch Exists? · Use Existing Branch · Record New Branch Baseline · Fetch Repository Tree

Bloc 2 — Lecture du code source (4 core + 5 extension)

Build Independent Repository Policy · Expand Finding Source Files · Fetch Finding Source Context · Expand Referenced API Sources · Fetch Referenced API Sources · Validate Source Context Completeness · Expand Required Dependency Sources · Fetch Required Dependency Sources · Validate Required Dependency Sources

Bloc 3 — Plan de correction (3)

Prepare Generic Remediation Plan · Generate Remediation Plan · Validate Generic Remediation Plan

Bloc 4 — Génération du patch, Boucle A (10)

Merge Effective File Results · Route Planned File Operation · Seed New Planned File · Fetch Repository Files · Prepare - Code Patch Body · de Patch - HTTP Request · Parse - Code Patch Output · Hash Candidate File Content · Accumulate Candidate File · Prepare Candidate Manifest

Bloc 5 — Contrôle mécanique (1)

Generic Candidate Preflight

Bloc 6 — Relecture par une 2^e IA, un seul fichier (2)

Independent Semantic Review · Enforce Independent Review

Extension — vérification multi-fichiers (16)

Classify Candidate Coordination Scope · Hash Cross-File Candidate Manifest · Validate Cross-File Candidate Manifest · Validate Cross-File Type Coherence · Prepare Cross-File COMPILE_MAIN · Call Cross-File COMPILE_MAIN · Enforce Cross-File COMPILE_MAIN · Prepare Cross-File COMPILE_TESTS · Call Cross-File COMPILE_TESTS · Enforce Cross-File COMPILE_TESTS · Prepare Cross-File FULL_TEST · Call Cross-File FULL_TEST · Enforce Cross-File FULL_TEST · Prepare Cross-File Review · Independent Cross-File Semantic Review · Enforce Cross-File Review

Bloc 7 — Vérification finale du candidat (11)

Hash Candidate Manifest · Assemble Candidate Manifest · Call Candidate Verification · Call Write Guard · Write Guard Passed? · Branch Existed At Generation? · Re-check Existing Branch Head · Re-lookup Baseline Ref Before Creation · Call Remote Head Drift Guard · Remote Head Drift Passed? · Branch Existed At Generation? (Pass 2 Entry)

Bloc 8 — Écriture Git & Pull Request, Boucle B (30)

Re-lookup Remediation Branch Before Create · Prepare Branch Creation · Create Missing Branch · Expand Manifest Files · Loop Over Manifest Files · Recompute Content Hash Before Send · Verify Content Hash Before Send · Candidate Creates File? · Create File in Branch · Update File in Branch · Build File Result (Pass 2) · Classify GitHub Write Error (Pass 2) · Transport Requires Read Back? (Pass 2) · Read Back File After Write Error (Pass 2) · Decode Reconciled Remote Content · Hash Reconciled Remote Content · Evaluate GitHub Write Reconciliation (Pass 2) · Remote Candidate Present? (Pass 2) · Lookup Head After Reconciled Write (Pass 2) · Build Reconciled File Result · Merge Effective File Results 2 · Validate Batch Completeness · Lookup Existing Batch PR · Select Existing PR · Existing PR? · Create Pull Request1 · Use Existing PR · Save Execution Result to Backend · If · Send an Email

Gestion d'échec / statut (73)

68 filets de sécurité "vivants" couvrant chacun des blocs ci-dessus (liste des 74 en section 3, dont le filet de `Return Policy Rejected`), + 4 gestionnaires spécifiques : Persist Verification Failure · Persist Base Moved Failure · Prepare WF2 Failure Status · Persist WF2 Failure Status

Nœuds morts / non câblés (17)

Jamais câblés (4) : Collect File Updates · Log Fetch Error · Persist File Update Failure · Persist PR Creation Failure
Ancienne chaîne désactivée (13) : Build File Result · Classify GitHub Write Error · Transport Requires Read Back? · Read Back File After Write Error · Evaluate GitHub Write Reconciliation · Remote Candidate Present? · Lookup Head After Reconciled Write · et leurs 6 filets de sécurité associés